

FP2MP-CORE · LANGGRAPH REACT AGENT

BlocksNet Agent

Архитектура агента городской аналитики

ReAct · LangChain

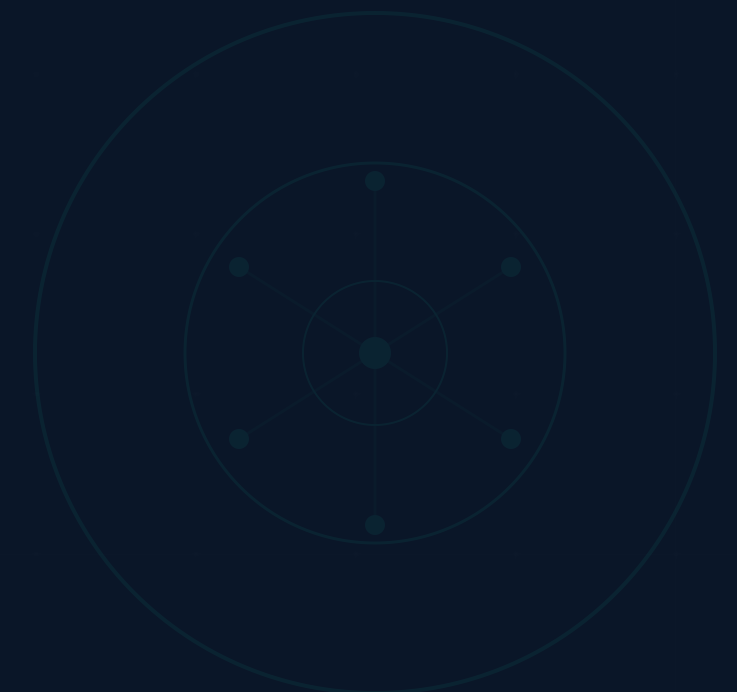
30+ инструментов

BlocksNet Library

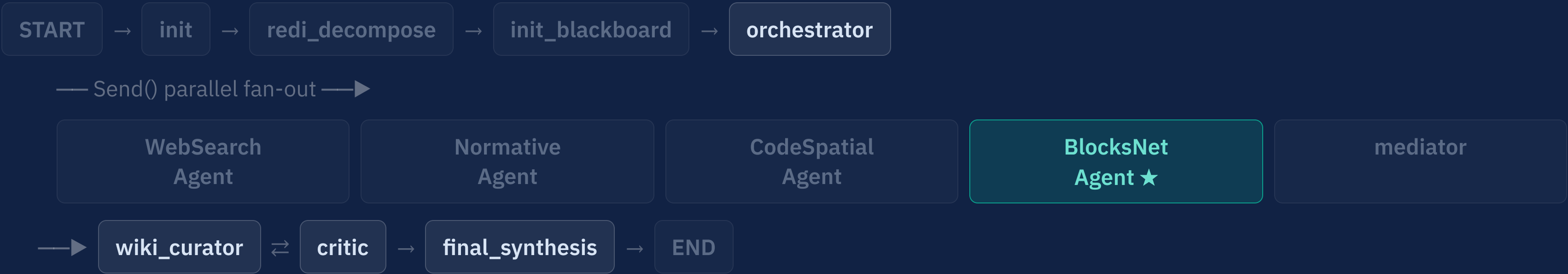
TPE Optimizer

ChromaDB

HuggingFace Embeddings



LangGraph StateGraph: 4 агента-исполнителя + Orchestrator → WikiCurator → Critic



BlackBoard: трёхуровневое состояние —
`raw_data` → `wiki` → `output`

Оркестратор выбирает агента через LLM по
сарабILITY-карточкам, не по жёсткому маршруту

WikiCurator продвигает факты ($\text{conf} \geq 0.78$) из `raw_data` в `confirmed facts`

Аналитик локальной городской модели

Что делает

- Вычисляет городские метрики по **предвычисленной модели**: обеспеченность сервисами, доступность, центральность, морфологию
- Генерирует и **измеряет** гипотезы развития через TPE-оптимизатор и независимую сценарную проверку
- Возвращает **структурированный вывод** с числами, block_id, уверенностью и ограничениями — машиночитаемый формат

Разграничение ролей с другими агентами

BlocksNetAgent ← эта презентация

Локальная модель: обеспеченность, доступность, плотность, оптимизация зон

CodeSpatialAgent

Свежие OSM / веб-геоданные, новые вычисления, объекты вне модели

WebSearch / Normative

Веб-контент, законы, СП, нормативные ограничения

351

кварталов в модели

46

типов сервисов

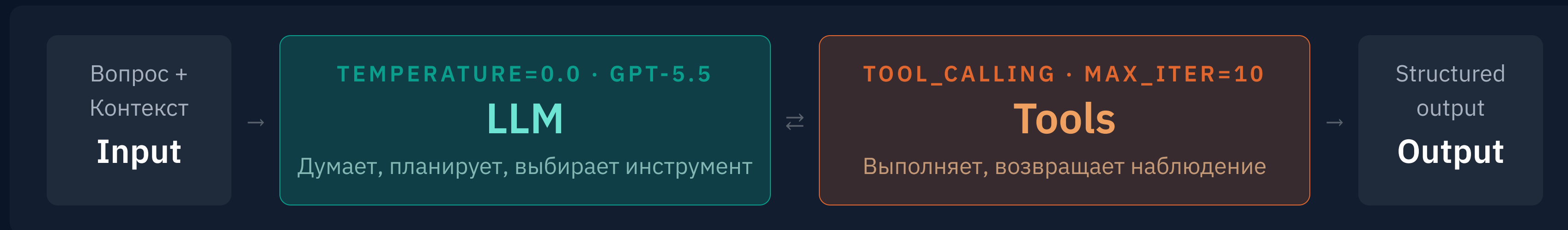
351²

матрица доступности

30+

инструментов агента

ReAct-агент поверх LangChain AgentExecutor



`create_tool_calling_agent`

Native tool_calling API — нет ручного парсинга Action/Observation строк.
Стандартный LangChain подход.

`return_intermediate_steps=True`

Все шаги (tool, input, observation) сохраняются, передаются в систему для логирования и scoring уверенности

`ChatPromptTemplate + .partial()`

system + human +
MessagesPlaceholder(agent_scratchpad).
Capability_cards инжектируются через .partial() до создания executor

`tool_state: dict`

Общий кэш для всех инструментов:
blocks, acc_mx, результаты compute_*.
Избегает повторной загрузки файлов.

`capability_cards (k=4)`

Перед запуском через ChromaDB
извлекаются 4 релевантных карточки и
инжектируются в system prompt

`verify loop`

Если в выводе обнаруживаются
unverified гипотезы — автоматически
запускается второй проход
AgentExecutor

8 обязательных правил работы

- 1 Данные загружаются автоматически. `load_blocks()` / `list_cached_data()` — для обзора и проверки кэша
- 2 **Не останавливаться после загрузки** — сразу вызывать нужные `compute_*` под задачу
- 3 Типы сервисов — только из `list_service_types()`. Нормативы `demand` и `accessibility` — из `list_key_services()`, не угадывать
- 4 PLAN должен ссылаться на CARD id из Retrieved capability cards. Если ни одна не подходит — вызвать `find_blocksnet_capability`
- 5 Все результаты автосохраняются в `data/model/output` — CSV и карты
- 6 При ошибке инструмента — объяснить проблему и продолжать; не отказываться от ответа
- 7 **Нельзя завершить без финального блока:** ANALYSIS PLAN / RESULT / REFLECTION / HYPOTHESES / NUMERIC SELF-CHECK / CONFIDENCE / LIMITATIONS
- 8 **ANALYSIS PLAN обязателен до `compute_*`:** вопрос → информационные потребности → метрики с обоснованием. Нельзя вызывать `compute_*` до объяснения зачем

Строгое ограничение области: работа ТОЛЬКО с локальной моделью `data/model`. Нельзя использовать OSM, веб или внешние геоданные. Если вопрос выходит за пределы — явно указать в LIMITATIONS.

Машиночитаемые секции — парсятся регулярными выражениями

ANALYSIS PLAN	Обязателен до расчётов: вопрос → информационные потребности → выбранные метрики и инструменты с обоснованием, ссылки на CARD id
RESULT	Ключевые числа модели с единицами и интерпретацией — основной машиночитаемый вывод агента
REFLECTION	Лучшие / худшие кварталы, дефициты обеспеченности, связь метрик — с цитированием чисел и block_id
HYPOTHESES	claim / test / verdict — каждый verdict обязан содержать число и block_id из фактического вывода инструмента; иначе: verdict: unverified
NUMERIC SELF-CHECK	Диапазоны, единицы, санити-проверка числовых результатов — явная метакогнитивная валидация агента
FOLLOW_UPS	Необязательные вопросы для следующих раундов с явным указанием: WebSearchAgent: / NormativeAgent: / BlocksNetAgent:
CONFIDENCE	0.0–1.0 — вычисляется автоматически вышестоящим кодом по числу успешных compute-вызовов, наличию reflection и верифицированных гипотез
LIMITATIONS	Что не учтено; явно отмечается выход за пределы data/model — список через «;» — парсится в массив limitations агентом

7 семейств инструментов · ~30 функций

 data	 network	 provision	 services	 indicators	 optimize	 viz · registry
load_blocks	compute_mean_...	compute_servi...	compute_svc_d...	compute_densi...	suggest_targe...	save_metric_m...
load_acc_matr...	compute_media...	compute_share...	compute_svc_c...	compute_dev_i...	propose_zone_...	 Capability Cards find_blocksne... ChromaDB HuggingFace embeddings
list_cached_d...	compute_max_a...	Пресеты: key · basic advanced · comfort	compute_svc_c...	build_adjacen...	optimize_zone...	
list_service_...	compute_conne...		compute_shann...	FSI · GSI MXI · OSR	compute_scena...	
list_key_serv...	compute_lu_acc		compute_svc_c...		TPE (Optuna)	
get_block_info	compute_area_...		compute_pop_c...			
get_analysis_...						

Данные модели и пространственная доступность

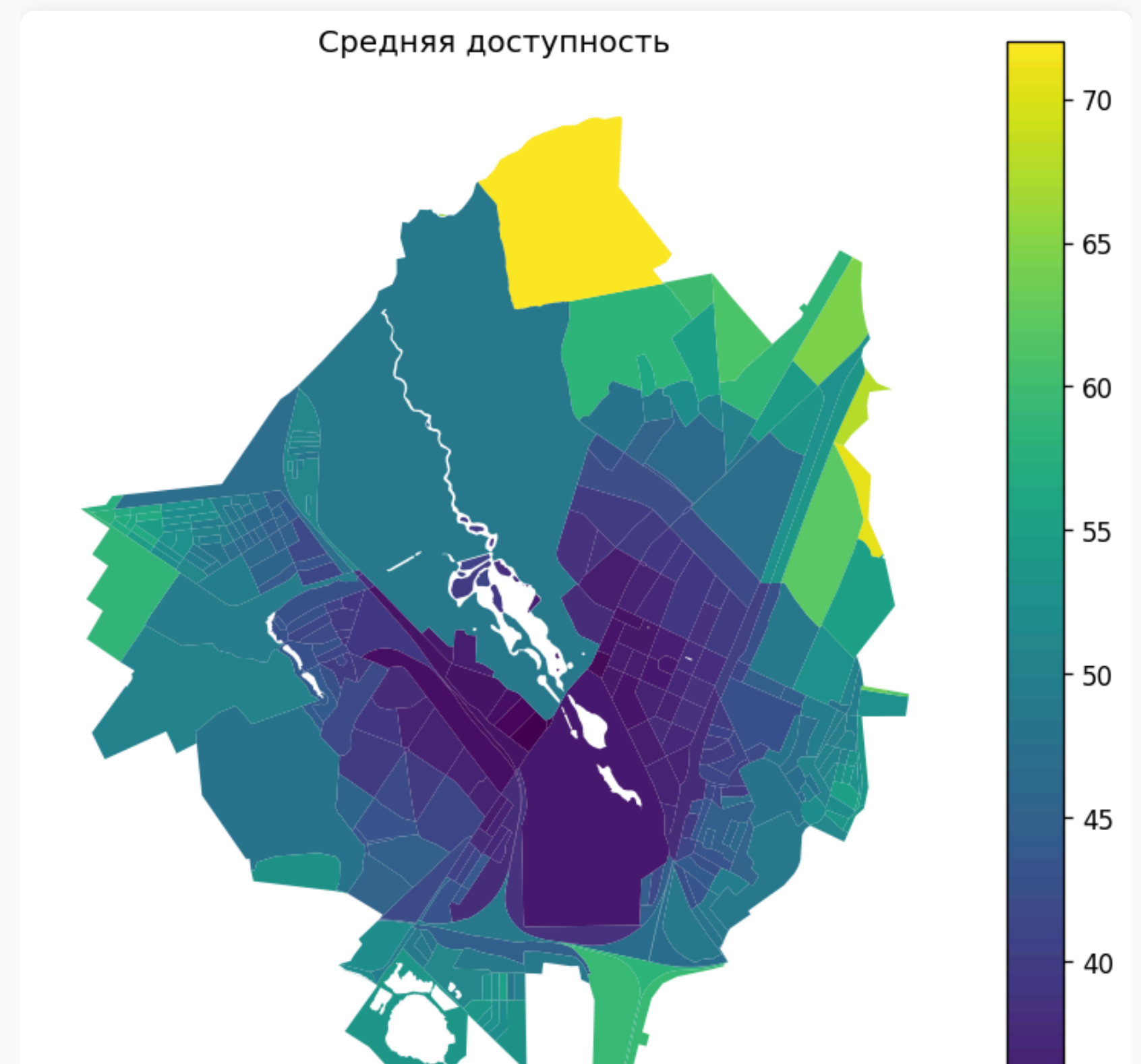
data — основа всего

- `load_blocks()` — GeoDataFrame 351 кварталов с сервисами, населением, типами землепользования
- `load_accessibility_matrix()` — матрица 351×351, диапазон 0.8–112.8 мин; кэшируется в `tool_state`
- `list_key_services()` — нормативные demand и accessibility из `service_type.json`
- `get_block_info(block_id)` — детальный профиль квартала для верификации гипотез

network — доступность по матрице

- `compute_mean_accessibility()` — среднее время до всех кварталов
- `compute_connectivity()` — индекс связности на основе распределения доступности
- `compute_land_use_accessibility(lu)` — доступность до зон нужного типа (RESIDENTIAL, BUSINESS...)

Карта: средняя доступность (Гатчина)



Конкурентная обеспеченность населения сервисами

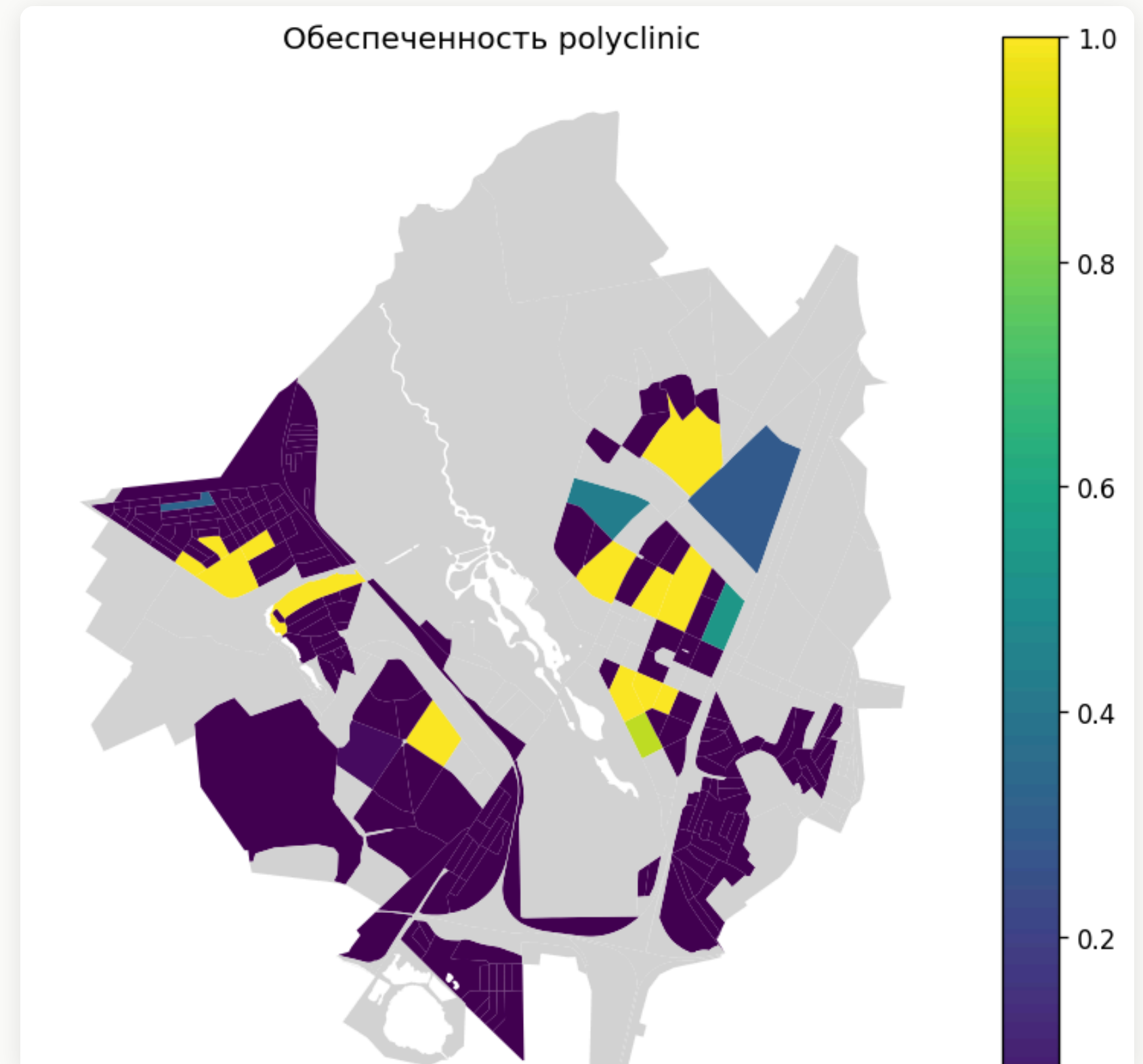
Алгоритм: `competitive_provision` (LP)

Каждый квартал конкурирует за ёмкости ближайших сервисов в радиусе `accessibility_minutes`. Задача LP-распределения решается итеративно до `max_depth`.

- **provision_strong** — доля населения с полным покрытием (≥ 1.0)
- **provision_weak** — доля с любым (включая частичным) покрытием
- Пресеты: `key` / `basic` / `advanced` / `comfort` — батч по набору сервисов с нормативами
- Результаты сохраняются в CSV и карте: `competitive_provision_{service}.csv`

```
compute_service_provision(  
    service_type="basic",  
    accessibility_minutes=15  
)  
→ service | strong | weak | full_blocks | missing
```

Пример: обеспеченность поликлиниками



Функциональный профиль и морфология кварталов

Сервисные метрики

`compute_services_centrality()`

Взвешенный индекс — находит городские центры и субцентры на основе ёмкости сервисов и доступности

`compute_population_centrality()`

Централизация населения по графу смежности — выделяет демографические центры нагрузки

`compute_shannon_diversity()`

Индекс разнообразия Шеннона по типам сервисов в квартале — маркер смешанной функции

`compute_services_density() · collocation()`

Плотность сервисов на кв.м площади квартала · матрица совместного расположения типов

Морфология застройки

`compute_density_indicators()`

FSI · GSI · MXI · OSR — плотность, пятно застройки, смешанность, открытое пространство

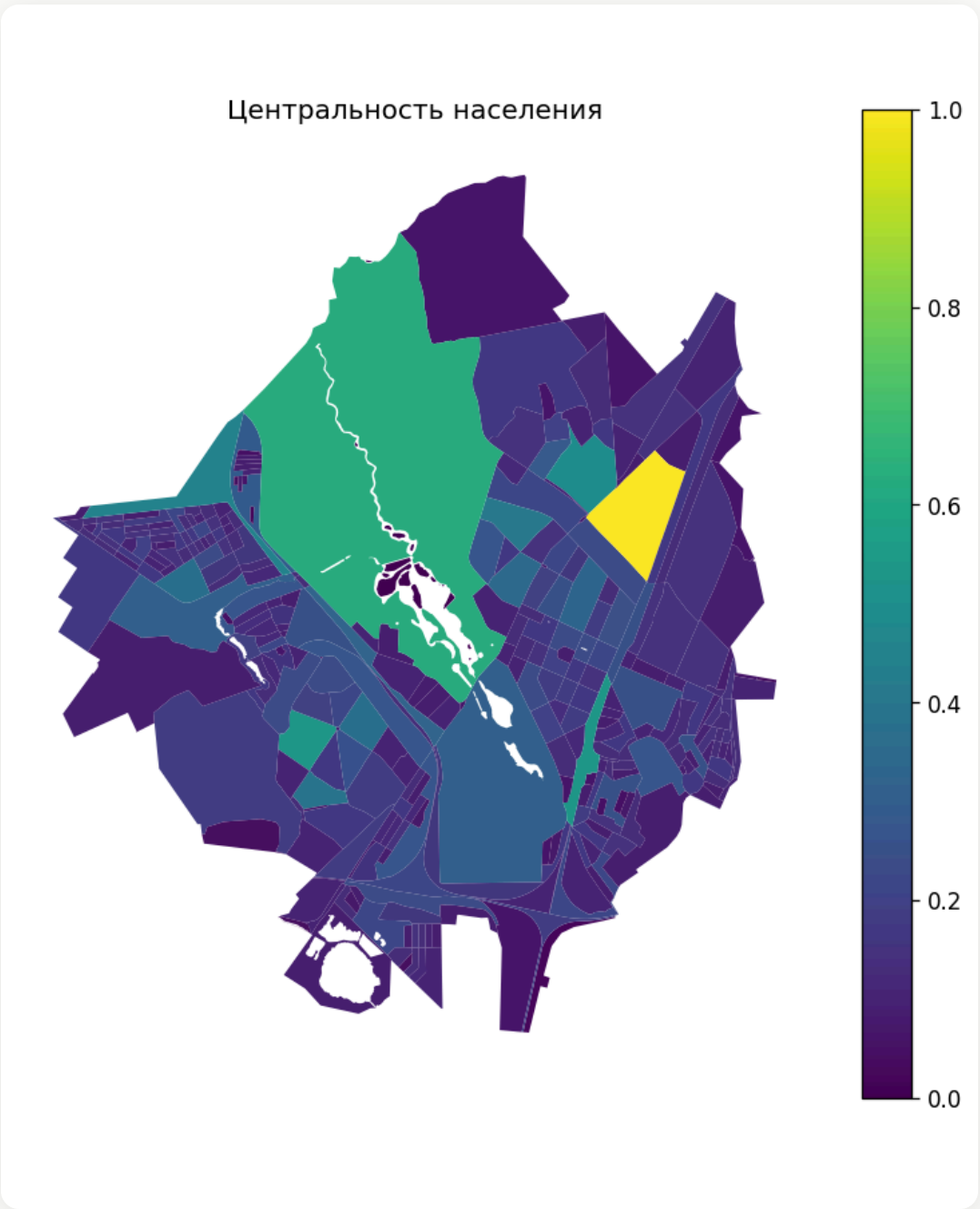
`compute_development_indicators()`

Освоенность территории: отношение застройки к открытому пространству

`build_adjacency_graph()`

Граф пространственной смежности кварталов — база для `population_centrality`

Центральность населения



Блок 331 — максимум 1.000

Петля гипотез: generate → test → verify

ШАГ 1

suggest_target_blocks

Отбирает кварталы-кандидаты по критерию из кэша метрик: low_provision, high_population или фильтр по land_use

```
criterion="low_provision"
```



ШАГ 2

propose_zone_development

TPE-оптимизатор (Optuna) подбирает состав и объём сервисов для выбранных block_ids. Оптимизирует взвешенную обеспеченность.

```
block_ids=[331, 108, 289...]  
service_set="basic"
```



ШАГ 3

compute_scenario_provision

Независимая верификация: пересчитывает обеспеченность before / after добавления ёмкостей из решения оптимизатора

Почему три шага?

Шаг 3 — независимый LP-пересчёт, не тот же optimizer. Исключает overfit к функции цели TPE.

Сохраняемые артефакты

solution_services.csv · solution_area.csv ·
delta_demand.csv · PNG-карта зоны
оптимизации

Правило в промпте

Порядок шагов выбирается по задаче, но generate и test — оба обязательны для гипотезы развития

Capability Cards — workflow-библиотека

- JSON-карточки в data/blocksnet_kb/ описывают готовые workflow для типовых аналитических задач
- **4 карточки** инжектируются в system prompt перед каждым запросом через семантический retrieval
- Retrieval: **ChromaDB** + paraphrase-multilingual-MiniLM; keyword-fallback если embeddings недоступны
- Правило промпта: PLAN должен ссылаться на CARD id. Если карточка не подходит — вызвать find_blocksnet_capability

Структура карточки

id / name	Уникальный идентификатор и название workflow
when_to_use	Условия применимости — используется при vector retrieval для подбора релевантных карточек
tools	Список инструментов, которые нужно вызвать для выполнения workflow
io_contract	Ожидаемые входы и выходы каждого шага workflow
prerequisites	Какие данные должны быть в tool_state до вызова инструментов
pitfalls	Типичные ошибки и как их избежать при работе с этим workflow

Автоматическая проверка гипотез и scoring уверенности

Петля верификации гипотез

1. Парсинг вывода агента

`_extract_sections() + _has_unverified_hypotheses()`

2. Если найдены **unverified** гипотезы

Запускается второй проход AgentExecutor с задачей закрыть именно их через `get_block_info / compute_*`

3. Merge результатов

`_merge_verified_hypotheses()` обновляет блок HYPOTHESES в финальном выводе

Требование к verdict

Verdict должен содержать **конкретное число** и **block_id** из результата инструмента. Verdict без выполненного теста — недопустим.

Scoring уверенности (confidence)

0.3 0 успешных `compute_*` вызовов
Нет инструментальных свидетельств — низкая надёжность

0.55 1–2 успешных инструмента
Частичное свидетельство — ниже порога продвижения

0.68 3+ инструментов, нет reflection
Хорошая доказательная база

0.78 3+ инструментов + reflection
Верхний порог — WikiCurator продвигает на BlackBoard в confirmed facts

unverified гипотезы ограничивают: `conf = min(conf, 0.62)`

КЕЙС-СТАДИ

ВХОДНОЙ ЗАПРОС

«Создай мастер-план города Гатчина»

Открытая постановка без эталонного ответа ·
gpt-5.5 · max_iterations=7

351

кварталов

83k

жителей в модели

46

типов сервисов

~300

вызовов инструментов

44

минуты прогона

BlocksNetAgent ★ conf=0.78

CodeSpatialAgent conf=0.30

WebSearchAgent

NormativeAgent

Mediator

Critic → #fu2

Реальный trace прогона: от диагностики к измеренной рекомендации

- 1** `load_blocks()`
351 кварталов, 83 320 жителей, 46 типов сервисов
- 2** `list_key_services()`
Нормативы demand и accessibility из service_type.json
- 3** `compute_service_provision("basic")`
Батч 7 сервисов: polyclinic=0.295, convenience=0.296, pitch=0.460
- 4** `compute_mean_accessibility()`
Среднее 45.59 мин; блок 196=32.67 (лучший), блок 285=72.01 (худший)
- 5** `compute_connectivity()`
Индекс связности транспортной сети по кварталам
- 6** `compute_services_centrality()`
Топ-5: блок 136=0.804, 135, 177, 3, 82
- 7** `compute_population_centrality()`
Макс: блок 331=1.000 — демографический центр города
- 8** `compute_density_indicators()`
FSI, GSI, MXI, OSR для 351 квартала → сохранено в density_indicators.csv
- 9** `suggest_target_blocks("low_provision")`
Кандидаты по дефицитам: [331, 108, 289, 87, 89, 9, 18, 2]
- 10** `propose_zone_development([331, 108...])`
TPE (300 сек) → solution_services.csv, delta_demand.csv, карта зоны
- 11** `compute_scenario_provision()`
Before/after: polyclinic 0.295→0.833, school 0.774→0.898
- 12** `get_block_info(331), get_block_info(9)...`
Детальный профиль кварталов для верификации HYPOTHESES с числами и block_id

Доступность и обеспеченность по локальной модели

Доступность

45.59

мин — среднее по городу

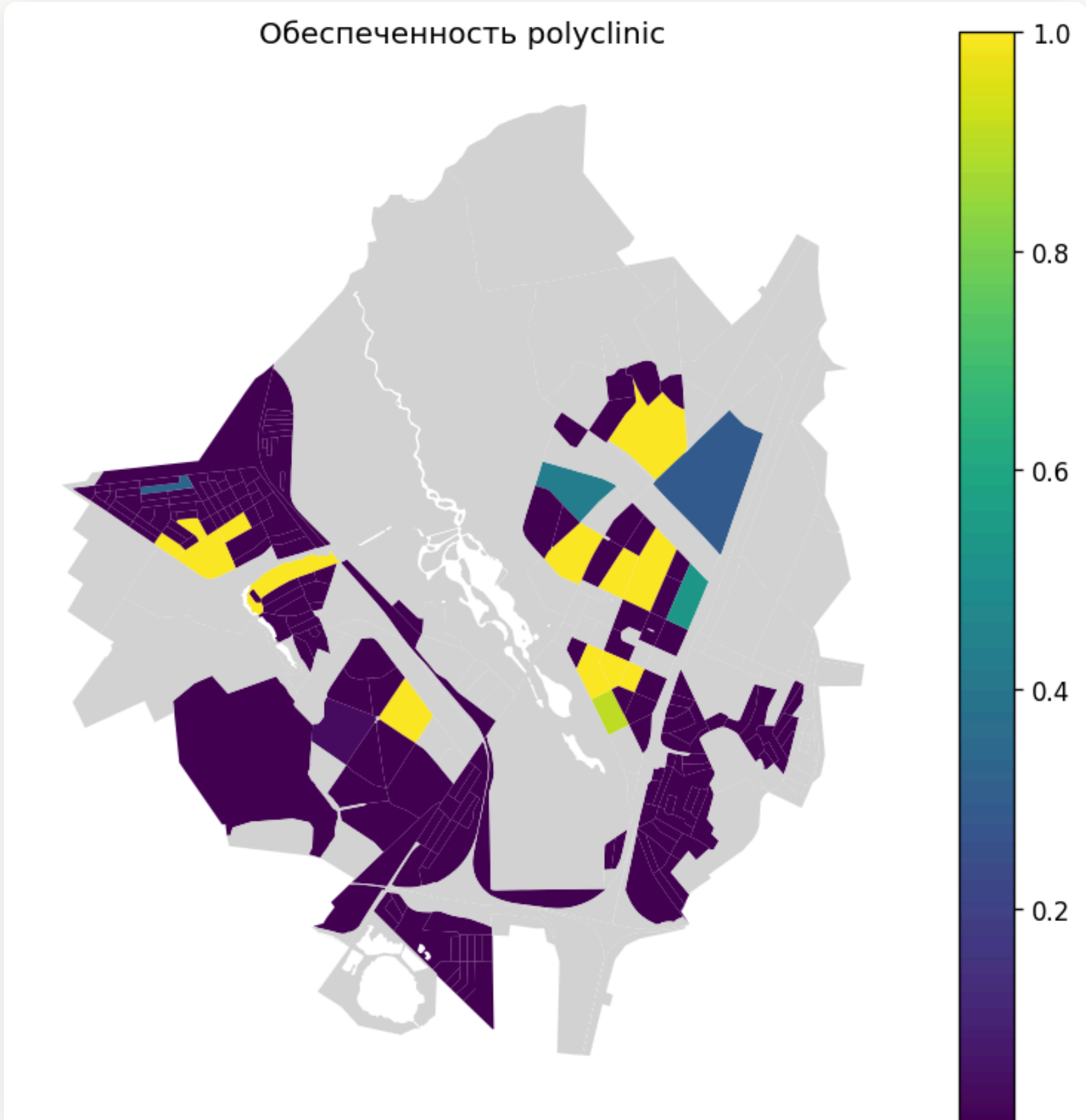
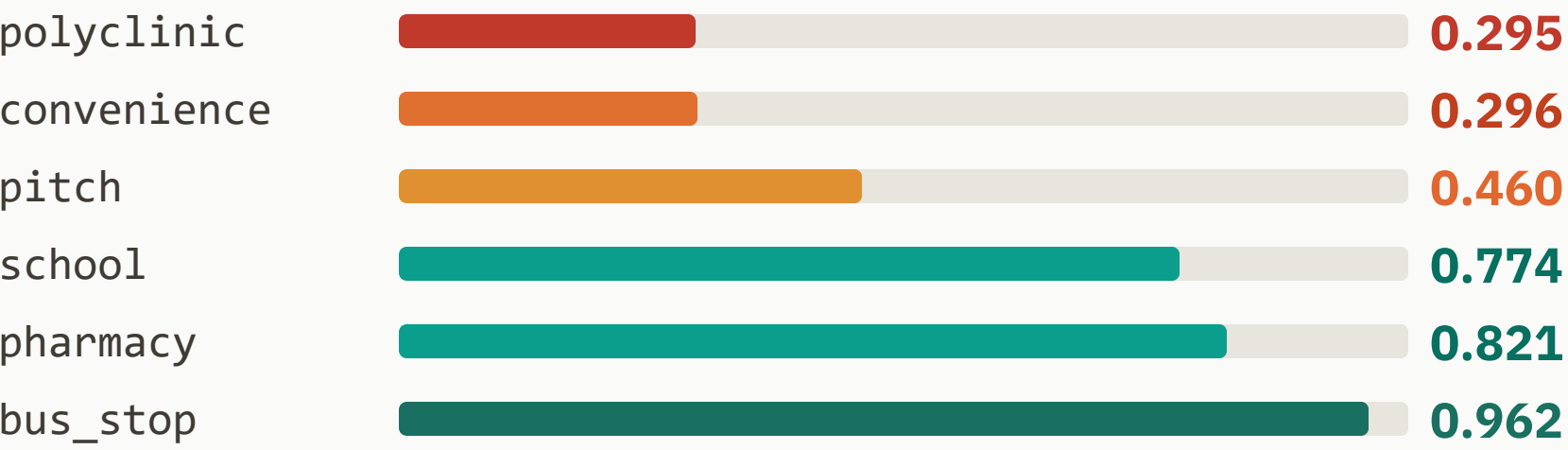
32.67

лучший — блок 196

72.01

худший — блок 285 (периферийная изоляция)

Обеспеченность (provision_strong)



Кварталы [331, 108, 289, 87, 89, 9, 18, 2] – before / after (provision_strong)

Сервис	До	После	Δ
polyclinic	0.295	0.833	+182%
school	0.774	0.898	+16%
pharmacy	0.821	0.900	+10%
kindergarten	0.613	0.710	+16%
pitch	0.460	0.545	+18%

Верификация: claim / test / verdict

claim: добавление ёмкостей в 8 кварталах улучшит обеспеченность

test: compute_scenario_provision(use_cached_solution=True)

verdict: supported — polyclinic 0.295→0.833, school 0.774→0.898

LIMITATIONS: стохастический TPE; результат зависит от выбранных block_ids и весов сервисов. Модельная гипотеза, не проектное решение.

Карта предложенных сервисов

